

Decisions

// ——— IMPORTS

```
import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
```

// ——— CLASS

```
class MainActivity : AppCompatActivity() {
```

// ——— onCreate()

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
```

```
    val tvResult = findViewById<TextView>(R.id.tvResult)
    val btnCheck = findViewById<Button>(R.id.btnCheck)
```

```
    btnCheck.setOnClickListener {
```

```
        // variable we use across all decision examples
        val age = 20
        val score = 85
        val day = "Monday"
        var output = "" // we build up this string to show all results
```

// ——— DECISION 1: if only

```
        // runs the block ONLY if the condition is true
        // if it is false, nothing happens — no else needed
        if (age >= 18) { // condition: is age 18 or more?
            output += "You are an adult\n" // only runs if condition is TRUE
        }

        // age = 20, so 20 >= 18 is TRUE → line runs
```

// if age was 15, 15 >= 18 is FALSE → nothing happens

// — DECISION 2: if / else

```
// if = runs when condition is TRUE
// else = runs when condition is FALSE
// one or the other ALWAYS runs — never both, never neither
if (score >= 50) {           // is score 50 or more?
    output += "Result: Pass\n"    // TRUE path
} else {
    output += "Result: Fail\n"    // FALSE path
}

// score = 85, so 85 >= 50 is TRUE → "Pass" runs
```

// — DECISION 3: if / else if / else

```
// use this when you have MORE than two possible outcomes
// Kotlin checks TOP to BOTTOM and stops at the FIRST true condition
// else at the bottom is the fallback if nothing matched
if (score >= 90) {
    output += "Grade: A\n"    // checked first
} else if (score >= 75) {    // only checked if first was false
    output += "Grade: B\n"    // score=85, 85>=75 TRUE → runs here
} else if (score >= 60) {    // only checked if above were false
    output += "Grade: C\n"
} else {                    // runs if ALL conditions above were false
    output += "Grade: F\n"
}
```

// — DECISION 4: if with AND / OR operators

```
// && means AND — BOTH conditions must be true
// || means OR — AT LEAST ONE condition must be true
// ! means NOT — flips true to false or false to true

if (age >= 18 && score >= 50) { // AND: both must be true
    output += "Adult who passed\n"
}
```

```

    if (age < 18 || score >= 50) {    // OR: at least one true
output += "Young OR passed\n"
    }

    val isLoggedIn = false
    if (!isLoggedIn) {                // NOT: flips false to true
output += "Please log in\n"          // runs because !false = true
    }

```

// — DECISION 5: when — match exact value

```

// when is Kotlin's version of switch
// checks the value in () against each branch top to bottom
// stops at the FIRST match, runs that branch, skips the rest
when (day) {
    "Monday" -> output += "Start of week\n" // matches! runs this
    "Friday" -> output += "End of week\n"
    "Sunday" -> output += "Rest day\n"
    else     -> output += "Normal day\n"    // fallback if nothing matched
}

```

// — DECISION 6: when — number ranges

```

// "in 90..100" checks if score falls inside that range (inclusive)
when (score) {
    in 90..100 -> output += "Distinction\n"
    in 75..89  -> output += "Merit\n"       // score=85, lands here
    in 60..74  -> output += "Pass\n"
    else      -> output += "Fail\n"
}

```

// — DECISION 7: when — multiple values per branch

// use a comma to match more than one value in the same branch

```

when (day) {
    "Saturday", "Sunday" -> output += "Weekend!\n" // either matches
    "Monday", "Tuesday",
    "Wednesday", "Thursday",

```

```

        "Friday"        -> output += "Weekday!\n" // any of these matches
    else                -> output += "Unknown day\n"
}

```

// — DECISION 8: if as an EXPRESSION

```

// instead of running code, if returns a VALUE into a variable
// cleaner than writing if/else just to assign something
val status = if (score >= 50) "Pass" else "Fail"
// status = "Pass" because 85 >= 50 is true
// same as: if(score>=50){ status="Pass" } else { status="Fail" }

val label = if (age >= 18) "Adult" else "Minor"
// label = "Adult" because age=20 >= 18 is true

output += "Status: $status, Label: $label\n"

```

// — DECISION 9: when as an EXPRESSION

```

// when can also return a value directly into a variable
val grade = when {
    score >= 90 -> "A" // no variable in () — just conditions
    score >= 75 -> "B" // score=85, this is true → grade = "B"
    score >= 60 -> "C"
    else       -> "F"
}
// grade now holds "B"
output += "Final grade: $grade"

// display all results in the TextView
tvResult.text = output
}
}
} // end of class

```